

Ad hoc Protocols with Code-as-Policy for Decentralized Multi-Robot Control

Anonymous Authors

Abstract—Robots deployed in the wild will often need to collaborate with robots they have never encountered before. Successful coordination may require going beyond high-level planning to establishing coupled low-level control to overcome partial observability, information asymmetry, or heterogeneity, all while staying safe and robust to asynchronous and dynamic deployment conditions. To address this, we present Protocols with Code-as-Policy (P-CaP). P-CaP enables robots to develop and deploy ad hoc communication protocols according to task demands, achieving coordinated control while remaining fully decentralized. Each robot relies on an LLM that first negotiates the message schema and content with its peers in natural language. Then, each robot implements its side of the agreed protocol directly in its control code. Once deployed, policies and protocols run inside each individual robot’s control loop, decoupled from the LLM, which keeps reasoning, renegotiating, and redeploying code as the task unfolds. We evaluate P-CaP on four asynchronous coordination tasks that involve private information and complementary sensing under partial observability. P-CaP matches or outperforms language-only and communication-free baselines on every task, solves time-critical situations, completes tasks faster than language-only coordination, and exhibits good performance when teams mix robots driven by different LLMs.

Website — <https://anon-researchers.github.io/capap.github.io/>

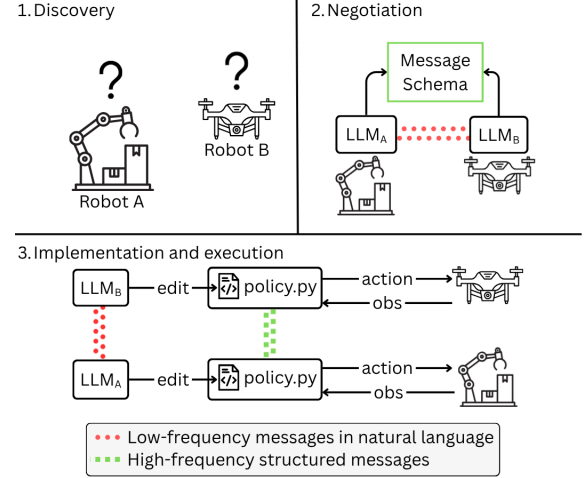


Fig. 1: The three phases of ad hoc multi-robot coordination with P-CaP: (1) robots receive a collaborative task at deployment; (2) at some point, they encounter each other and start to negotiate in natural language, agreeing on some ad hoc communication protocol to succeed; and (3) they independently implement the protocol, augment their control policies and execute them with the message exchange(s) in the loop.

I. INTRODUCTION

When humans collaborate on a task they rely on protocols, i.e., mutual agreements on how, when and what information to share, and they revise these agreements as the task unfolds. Generalist robots, which build on the generalization capabilities of language models [1] to reason about new tasks and environments [2], [3], will soon face similar situations. They will increasingly share their workspace and collaborate with other robots that differ in embodiment, sensing, and reasoning capabilities. However, generalist robots are typically trained to act alone [4] or within teams co-designed in advance [5]–[7]. Robots that meet during deployment must instead form protocols on the spot, for in many tasks the information they exchange shapes each robot’s actions. The challenge we address here is how robot teams can form, and keep updating the ad hoc communication protocols needed to coordinate.

For LLM-enabled robots, natural language offers a medium for such ad hoc coordination, since it requires no prior agreement beyond the language itself. Prior work has exploited it for joint planning [5], task allocation [6], reasoning over heterogeneous capabilities [7], and overcoming partial observability [8]. In most of the settings of these papers, the information needed for coordination can be exchanged in plain language in a few non-time-critical rounds of communication. Many coordination problems in robotics will, however, find

this assumption too restrictive: natural language, combined with the high inference latency of LLMs, would be ineffective at feeding information directly into a real-time control loop. One solution, then, is to implement structured task-specific messaging schema with appropriate ‘fields’ that the robots can populate and exchange. Since we do not know what the task will be, we ask whether such protocols can be implemented by the robots ad hoc.

In this work, we present *Protocols with Code-as-Policy* (P-CaP), a framework that enables robots in a team to dynamically implement new ad hoc communication protocols to run separately from their inter-LLM message exchange in a decentralized manner. P-CaP builds on the capabilities of Code as Policies (CaP) [9] by exposing a send/receive interface to the generated policies. Each protocol takes the form of a pair of programs, co-developed by the robots, that exchange structured messages at every control step (Section IV). To isolate the challenge of ad hoc protocol formation, we evaluate P-CaP’s zero-shot real-time performance in a set of four asynchronous coordination tasks designed to elicit the need for rapid exchange of private or complementary information (Section V). Across four LLM backbones in cross-play and against two baselines, we show that P-CaP is able to facilitate ad hoc development of fully asynchronous protocols that make a decisive difference in coordination and task completion, both

in simulation and real experiments.

II. RELATED WORK

Our work lies at the intersection of three lines of research: coordination with previously unseen teammates, the generation of robot control policies as code by LLMs, and LLM-based multi-robot coordination.

Ad hoc coordination. Ad hoc teamwork tackles agents collaborating with previously unseen teammates [10] and in the context of multi-robot coordination with LLMs has been approached by defining agentic patterns to help reason over signals from peers [11]. Zero-shot coordination [12], [13] poses as similar problem to ad hoc teamwork where the agents are also expected to coordinate with unseen agents but are assumed to have shared a training procedure. Related to both of these regimes, this paper tackles ad hoc coordination with unseen agents where we assume all robots to share the ability to communicate in natural language and generate policies programmatically.

Generating control policies with LLMs. We build on Code-as-Policy [9], which queries an LLM to write reactive robot policies as code, and the related ProgPrompt [14], which generates executable plans in code form. Later works refine this paradigm for a single robot. CaP-X [15] varies the abstraction level of the primitives exposed to the coding agent, obtaining the best performance with high-level, human-authored ones, and Graph-as-Policy [16] generates several coding agents to create a single-robot computation graph spanning perception, planning, and control. In both, the program governs a robot acting alone. P-CaP instead grants a single coding agent a peer-to-peer interface, so it implements communication protocols alongside the policy.

Coordination in LLM-based multi-robot systems. LLMs have proved effective at high-level planning for robot teams, covering task decomposition, task assignment, and reconciling heterogeneous capabilities [5]–[7], [17], [18], yet many such systems are centralized or hierarchical [6], [7], [17]–[19], sidestepping the consensus problem of decentralized teams [20], or abstract control away, leaving fixed planners or controllers to execute the primitives the LLM selects. Robots that meet during deployment are decentralized by construction, and the coordination they need lies at the level of actuation, not the plan. Closer to us are methods in which the LLM shapes control: GenSwarm [19] generates and refines a code-based policy for a swarm, as does LLM2Swarm [21] from a central LLM, with a per-robot LLM for inter-robot dialogue; COMPASS [22] refines skills through Code-as-Policy on each robot of a decentralized team, but communicates through a predefined mechanism; CaPE [23] edits paths for decentralized planning from peer conversations; and CHORUS [24] fine-tunes a VLA on multi-robot data to perform collaborative tasks communication-free. Our paper builds capabilities common with many of these methods but never before instantiated in one approach. Specifically, P-CaP adopts the code-as-policy approach for flexible control over the robots actions and

enables ad hoc, decentralized teams of robots to establish custom, task-specific protocols during deployment.

III. PROBLEM FORMULATION

We consider N independent, potentially heterogeneous robots that encounter each other during deployment and must cooperate to accomplish a task. The robots have been developed separately and have neither interacted nor coordinated with each other before this encounter.

Robots. Each robot $i \in \mathcal{N} = \{1, \dots, N\}$ has its own embodiment and perception stack, characterized by an observation space $\mathcal{O}^i$ and an action space $\mathcal{A}^i$. The robots operate in real time $t \in \mathbb{R}_{\geq 0}$, but each robot perceives and acts only at its own discrete instants $\{t_k^i\}_{k \in \mathbb{N}}$, which are not synchronized across robots. At each instant t_k^i , robot i receives a local observation $o^i(t_k^i) \in \mathcal{O}^i$, which reveals only part of the environment state $s(t) \in \mathcal{S}$, and applies the action

$$a^i(t_k^i) = \pi_{t_k^i}^i(h^i(t_k^i)) \in \mathcal{A}^i, \quad (1)$$

where $h^i(t)$ is the history of observations, actions, and received messages available to robot i up to time t , and π_t^i is the control policy active at time t . The environment evolves under the actions of all robots according to dynamics that are unknown to them. Each robot is also equipped with an LLM that can reason about the task, communicate with peers, and replace the active policy during deployment, which is why π_t^i may change over time. Each LLM query incurs a latency of at least $\lambda > 0$, which is typically much larger than the control period $t_{k+1}^i - t_k^i$.

Communication protocols. The robots can exchange messages through a peer-to-peer channel, which we assume introduces negligible delay. A message $m^{i \rightarrow j}(t) \in \mathcal{M}$ sent by robot i to robot j at time t , where $\mathcal{M}$ comprises both natural-language and structured messages, becomes part of h^j upon reception. Formalizing the notion introduced in Sec. I, we define a communication protocol from robot i to robot j as the pair $\mathcal{P}_t^{i \rightarrow j} = (\mathcal{M}_t^{i \rightarrow j}, C_t^{i \rightarrow j})$, where the message schema $\mathcal{M}_t^{i \rightarrow j} \subseteq \mathcal{M}$ specifies what information is shared and how it is encoded, and the sending rule $C_t^{i \rightarrow j} : h^i(t) \mapsto \mathcal{M}_t^{i \rightarrow j} \cup \{\emptyset\}$ specifies when a message is sent, with $\emptyset$ denoting that no message is sent. A protocol is useful only if the receiving policy π_t^j interprets the messages in $\mathcal{M}_t^{i \rightarrow j}$ consistently with the sender's intent. Each protocol is therefore realized jointly by the sending rule of one robot and the policy of the other, and both robots must agree on it.

Tasks. A task τ is specified to the robots through a natural-language description ℓ_τ at deployment time. Let ξ denote the joint trajectory of states, observations, actions, and messages over an episode, which starts at $t = 0$, when ℓ_τ is provided, and lasts at most $T_{\max}$. The outcome of an episode is given by a success predicate $\varphi_\tau(\xi) \in \{0, 1\}$, which encodes both task completion, e.g., all robots reaching their goals, and safety constraints that must hold throughout the episode, e.g., the absence of collisions. For successful episodes, we denote by $T_\tau(\xi) \leq T_{\max}$ the completion time. Since initial conditions

and the timing of events are randomized, we assess a team by its success expectation $\mathbb{E}[\varphi_\tau(\xi)]$.

Prior knowledge. At $t = 0$, each robot knows the task description ℓ_τ , its own spaces $\mathcal{O}^i$ and $\mathcal{A}^i$, that it may encounter peers with whom it has to cooperate, and the concept of a communication protocol as defined above. It knows neither the embodiments, observation spaces, and policies of its peers nor which information they hold, and it receives no indication of which protocols to establish or how to implement them. Consequently, message schema, sending rules, and the policies that consume the messages must all be created during the episode, while the task is already running.

Problem statement. Consider N robots and a task τ . Each robot i must construct online and in a decentralized manner, i.e., using only its prior knowledge and $h^i(t)$, a control policy π_t^i , message schemas $\{\mathcal{M}_t^{i \rightarrow j}\}_{j \neq i}$ and sending rules $\{C_t^{i \rightarrow j}\}_{j \neq i}$ such that the resulting protocols and policy solve within a single episode

$$\max_{\{\pi_t^i, \mathcal{M}_t^{i \rightarrow j}, C_t^{i \rightarrow j}\}_{i \in N, j \neq i}} \mathbb{E}[\varphi_\tau(\xi)]. \quad (2)$$

IV. PROTOCOLS WITH CODE-AS-POLICY (P-CAP)

The objective posed in (2) requires each robot to construct both a control policy π_t^i , and a set of message schemas and sending rules $(\mathcal{M}_t^{i \rightarrow j}, C_t^{i \rightarrow j})_{j \neq i}$, while the task is already running and using local information only. P-CaP realizes this as a single program that the robot writes for itself and executes locally. We build on Code-as-Policy [9], where an LLM synthesizes the control policy of a robot as code, and extend this paradigm in two directions. First, the generated program is given a peer-to-peer communication interface, so that it implements $(\mathcal{M}_t^{i \rightarrow j}, C_t^{i \rightarrow j})$ within the policy π_t^i and consumes the messages of its peers inside the control loop. Second, the LLM that writes the program stays online for the entire episode, and converses with peers to revise and redeploy the program as the task unfolds.

This leads to the architecture depicted in Fig. 2, where every robot runs two loops concurrently at different frequencies. The **low-frequency reasoning** loop is where the LLM perceives the environment, negotiates with peers in natural language, and writes and deploys code; it advances once per LLM query and is therefore bounded by the latency λ . The **high-frequency control** loop is where the deployed program runs; it advances the robot's control step $\{t_k^i\}_{k \in \mathbb{N}}$, and produces the actions $a^i(t_k^i)$ and the protocol messages $m^{i \rightarrow j}(t_k^i) = C_{t_k^i}^{i \rightarrow j}(h^i(t_k^i))$. This separation of loops is what makes the two requirements of Section III compatible: *what* to share and *how* to encode it requires reasoning and is slow, whereas *acting* on what has been agreed upon must happen at every control step. Between two consecutive deployments, policy and protocol run unchanged at the control rate until the robot decides to replace them.

A. The low-frequency reasoning loop

The reasoning loop begins at the Prompt Assembly module, which maintains the LLM's view of the history $h^i(t)$. The

environment streams local observations o^i to the robot as JSON objects, and Prompt Assembly aggregates this stream, together with the natural-language messages received from peer LLMs, into the prompt of the next query. It also gathers the feedback returned by other modules of the system: the outputs of executed commands, the contents of the files the LLM keeps open, the status, errors, and logs reported by the Policy Worker, and the notes the LLM has appended to its own memory. The LLM therefore reasons over a *textual* rendering of $h^i(t)$ that combines what the robot perceives, what its peers have told it, and what its own software is doing.

The LLM is also given a filesystem and a bash terminal to execute any command on the robot's operating system. At the start of an episode the filesystem holds a skill library of Python modules and a single deployable script, `policy.py`. The skill library encodes the prior knowledge granted in Section III, and endows the robot with the basic capabilities of navigation, collision avoidance, maintaining beliefs over partially observable quantities, parsing messages, and reacting to basic environment signals.

The `policy.py` file is the robot's interface to its own embodiment, and is the single artifact in which π_t^i and $\{C_t^{i \rightarrow j}\}_{j \neq i}$ are jointly expressed. The file follows a fixed skeleton: it consumes the observations available to the robot, returns an action in $\mathcal{A}^i$ at every control step, and may read from and write to the peer channel. The LLM can deploy a library skill unchanged, or import and remix skills into a new program.

The LLM is queried sequentially, and each query returns a set of blocks that the Action Parser dispatches according to their type. We allow various block types to let the agent *create* files, *edit* existing code, *read* files into or *drop* them from context, *run* and manage long-running jobs through the terminal, *append* information to its memory, *send* a message to a peer, and request the *deployment* of the current `policy.py`. These actions produce feedback that flows back to Prompt Assembly, so that the next query will contain the effects from the current step. This enables to diagnose internal events such as a failed deployment, and external events reported by its observations or peers.

B. The high-frequency control loop

This loop is handled by the Policy Worker, which runs asynchronously from the rest of the system and only acts when the LLM explicitly requests it. The agent iterates and drafts changes to `policy.py` without disrupting the controller that is currently driving the robot. The Policy Worker also wraps the implemented code with other environment libraries (such as ROS¹) that need not be edited by the LLM agent.

Once deployed, the program is executed at the control rate. At each instant t_k^i it receives a local observation $o^i(t_k^i)$ and messages the peer policies have sent through the channel, updates its internal state, and emits both the action $a^i(t_k^i) = \pi_{t_k^i}^i(h^i(t_k^i))$ from (1) and the messages prescribed by its sending rules $C_{t_k^i}^{i \rightarrow j}(h^i(t_k^i))$. Policy and protocol are

¹<https://www.ros.org>

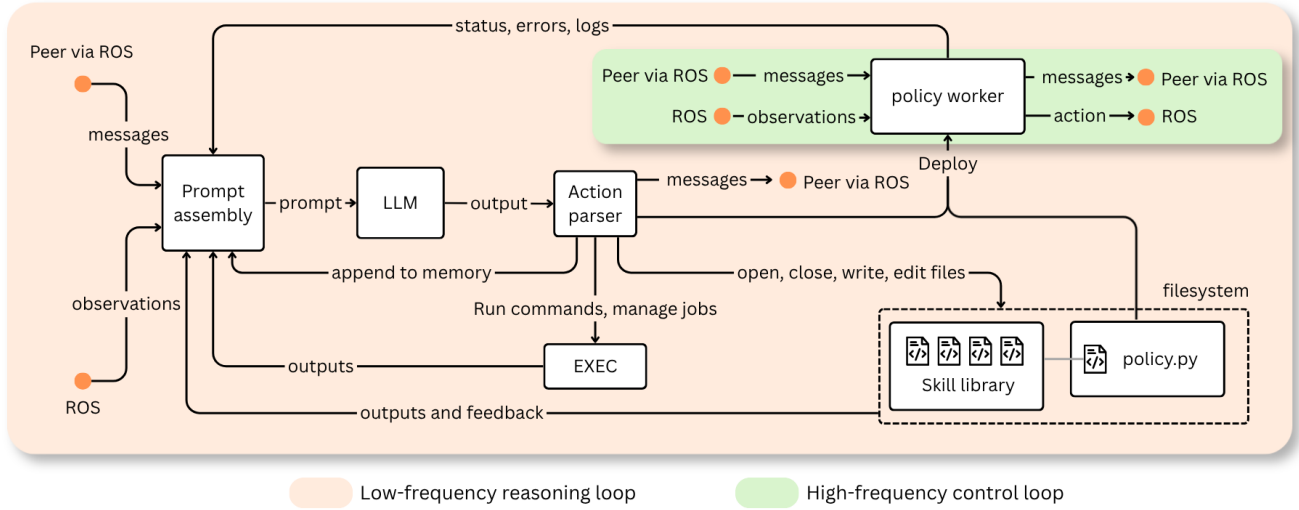


Fig. 2: Architecture of a single P-CaP robot. Two loops run concurrently at different cadences. In the low-frequency reasoning loop (orange), the Prompt Assembly module renders local observations, messages from peer LLMs, and the feedback returned by the other modules into the next prompt; the LLM replies with fenced blocks that the Action Parser dispatches to the peer channel, the memory, the terminal, the filesystem, or the Policy Worker. In the high-frequency control loop (green), the Policy Worker executes the deployed `policy.py` at the control rate, consuming observations and messages from peer policies and emitting actions and protocol messages, while reporting status, errors, and logs back to the reasoning loop. The two peer channels are kept separate: LLMs exchange natural language with peer LLMs, and deployed policies exchange structured protocol messages with peer policies.

executed by the same program within the same step, and thus, any delay between an event being observed by one robot and being acted upon by another is entirely due to the control period and the channel latency, not by the LLM latency λ .

C. Forming protocols as code

A protocol $\mathcal{P}_t^{i \rightarrow j} = (\mathcal{M}_t^{i \rightarrow j}, C_t^{i \rightarrow j})$ cannot be created by one robot alone. As stated in Sec. III, the sending rule belongs to robot i while the interpretation of its messages is embedded in the policy π_t^j of robot j , and the two are useful only if they agree. P-CaP builds them by combining, simultaneously, agreement in natural language with implementation in code.

Regarding agreement, the robots converse through the message blocks of their LLMs. They describe their observations, actuation and their requirements from their peers in order to act, and propose candidate schemas. They also converge on the fields, units, and semantics of the messages they will exchange, as well as on the conditions that should trigger them. This conversation is what fixes the schema $\mathcal{M}_t^{i \rightarrow j}$. Regarding implementation, each robot writes its own half of the agreement. The sender edits its `policy.py` so that its program computes and emits messages in the agreed schema, thereby realizing $C_t^{i \rightarrow j}$. Likewise, the receiver edits its own `policy.py` so that it parses those messages and routes their content into its control logic, thereby realizing the corresponding π_t^j . A protocol exists only as a pair of independently written programs whose behavior matches, and each robot verifies that match at runtime, from the messages it receives and from the effect they have on the task.

Consistently with the two kinds of messages that make up $\mathcal{M}$ in Sec. III, the natural-language traffic and the structured traffic take separate routes: messages produced by an LLM are delivered to peer LLMs, and messages produced by a deployed

policy are delivered to peer policies. This separation keeps the context window of the LLM free of the stream of the structured messages, and it lets each robot reason about the protocol in the abstract while its program executes it. The LLM remains aware of how the protocol is performing through local feedback from the Policy Worker and through peer reports in conversation, rather than by reading the traffic itself.

D. Asynchrony, revision, and safety

The two loops of Fig. 2 are never synchronized, neither within a robot nor across the team. Within a robot, the LLM is called only after the previous call has returned and the jobs it launched have terminated, while the deployed policy and protocol keep running at a fixed frequency throughout. Across the team, each robot negotiates, writes, and deploys at its own instants, so one robot may be revising its program while another is already executing the agreed protocol. Revising a protocol is therefore nothing more than another deployment: when a robot reports an event that the current agreement does not cover, the LLMs renegotiate mid-episode and each side swaps in a new program, without either robot stopping or having to wait for the other. This constitutes a closed-loop system with a piece-wise controller, in which transitions between policies happen when the system decides to load a new program as policy.

Because a policy is running at all times, the robots are deployed from $t = 0$ with a safe default policy, so that π_t^i is defined before any code has been written and the robot is responsive while its LLM is still reasoning or negotiating. The Policy Worker supervises the deployed program and reverts to this safe default policy in the event of a code-engineering failure. This ensures that the robot retains basic responsiveness while the LLM repairs the fault.

V. EVALUATION SETUP

A. Task suite

We evaluate P-CaP on four tasks (described below) that instantiate the problem of Sec. III while inducing different reasons for a protocol to be needed. The goal is to be able to isolate the role of ad hoc communication for real-time coordination of robot teams at first-time encounters. The tasks are defined in a 2D environment with two rectangular holonomic-drive robots acting in real time under partial observability. The task τ requires every robot to reach some waypoints without colliding with obstacles, moving objects, or peers. Layouts, initial poses, goals, and event times are randomized across seeds. At every tick, the environment emits a JSON observation with noise-free localization of the robot and of the objects it perceives, described by their vertices. The tasks, depicted in Fig. 3, are as follows:

Private Alarm. Each robot perceives its own private alarm signal while navigating to its goal, and the entire team must stop whenever any robot sees a red alarm. An episode succeeds if all robots reach their goals and no robot is in motion Δt_{grace} seconds after *any* red signal turns on.

Crossed Waypoints. Each robot must visit a sequence of waypoints that only its peer perceives. The waypoints are reshuffled every t_{shuff} seconds such that delayed information becomes expired and the robot misses the waypoint.

Blind Guidance. A ground robot must navigate a cluttered room without sensors, while an aerial robot with mapping capabilities hovers above it. The episode succeeds when the ground robot reaches its goal.

Blind Guidance with hazards. As above, with moving hazards that the ground robot must also avoid, so that guidance has to be issued continuously rather than once.

The first two tasks require exchanges of private information, while the last two need fusion of complementary sensing.

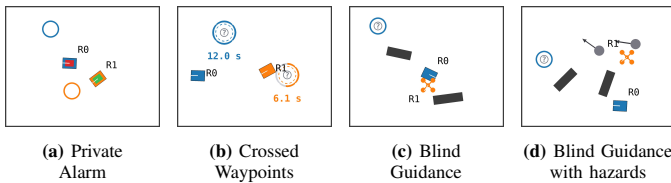


Fig. 3: Sample mid-episode snapshots from the four coordination tasks. In each task, holonomic ground robots or aerial drones must form a protocol to overcome partial observability.

B. Baselines and experimental setup

We use two baselines that represent the body of work discussed in Sec. II. The *language-only* baseline (CaP with communication) grants the agents the same Code-as-Policy capabilities as P-CaP, but withholds the peer-to-peer interface from the generated program, so that every exchange must pass through the LLM-to-LLM channel. The *communication-free* baseline (CaP) removes communication altogether, leaving each robot to act on its local observations with any policy

it wishes to write. We deliberately omit a centralized baseline that prescribes a protocol to the team because such an architecture would dissolve the problem we study. However, to ground any results, we also include a *handcrafted* decentralized protocol that we develop offline for each task separately.

All evaluations run in real time using cloud inference, so that λ is the latency a deployed system would actually suffer. We use four LLM models from different providers: Gemini 3.8 Flash [25], Claude Sonnet 5 [26], Muse Spark 1.3 [27], and DeepSeek V4.1 Flash [28]. Every task-backbone pair is run in self-play over five seeds with $T_{\text{max}} = 6$ min. Episode success is zero-shot: any safety violation classifies the episode as failed, but the episode continues until the goal is reached or until T_{max} expires. For the cross-play study of Sec. VI-C, each heterogeneous pair of backbones is run over three seeds (to keep the number of real-time cloud-inference episodes tractable). We report the empirical success rate $\mathbb{E}[\varphi_{\tau}(\xi)]$ and the completion time $T_{\tau}(\xi)$ over these seeds, with the latter split into the time taken for agreement and implementation, and the time spent executing it. Failures for an episode are classified by inspecting the corresponding exchanged messages, the reasoning traces, and code snapshots.

VI. RESULTS

A. Protocols make the difference

In Figure 4, we report the task success rates and completion times for each set in the evaluation suite. We find that P-CaP matches or outperforms both baselines on every task and every backbone, and competes very close to the hand-crafted implementation. Except in *Blind Guidance*, neither baseline succeeds in a single episode, whereas P-CaP solves each of these tasks in 5/5 episodes with the best LLM model. CaP fails in all tasks, confirming that the task suite requires policies that use more than local observations alone to satisfy the task goals and constraints. CaP with communication shows some success on *Blind Guidance* (3/5, compared to P-CaP's 5/5), the only task whose guidance can be delivered as a handful of verbal instructions.

Choice of LLM models. We also observe that performance varies markedly across LLM models, and none is uniformly strong: Gemini 3.8 Flash solves every task in at least 4/5 episodes, while DeepSeek V4.1 Flash collapses on both *Blind Guidance* variants. The LLM module is therefore a decisive design choice, since it determines whether a team forms a working protocol at all, not merely how quickly it does so.

Protocol Generation and Recovery from Mistakes. The handcrafted protocol sets a reference for task completion time. While P-CaP episodes last between ≈ 150 s to 320 s (♦ in Figure 4), a substantial fraction of that is spent in agreement and implementation; measured from the last deployment, the generated protocols for the strongest backbones complete the tasks within a few seconds of the handcrafted ones (● in Figure 4). The team therefore pays a one-off price to reach an agreement, and then runs a protocol as efficient as a handcrafted one. We let episodes continue until the goal is reached or T_{max} expires, so a team has a chance to renegotiate and

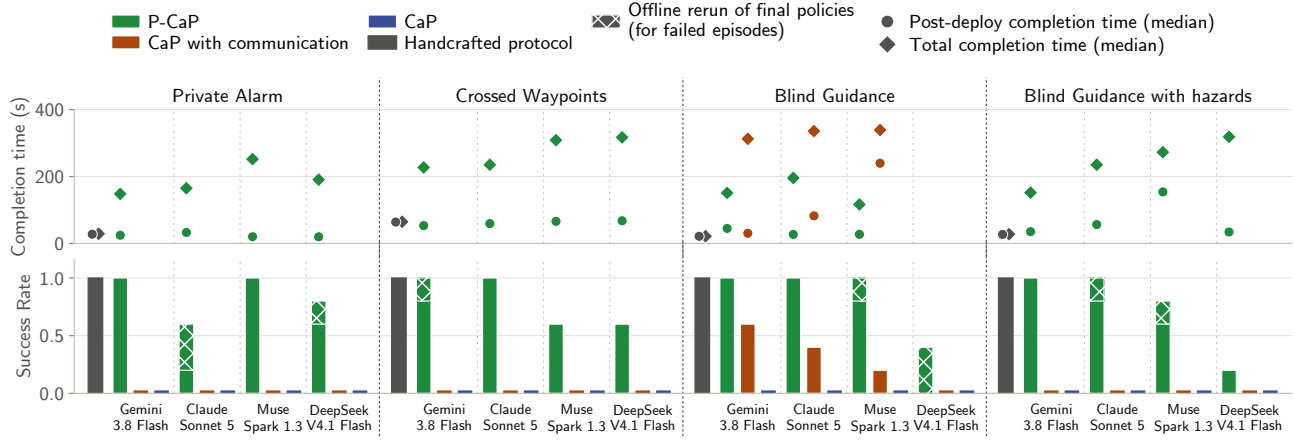


Fig. 4: Success rates and completion times of all evaluated methods on the four tasks of the suite. With the strongest backbones, P-CaP solves the tasks nearly as successfully as the handcrafted oracle protocol, whereas both baselines fail almost everywhere. Bars (bottom panel) give the fraction of successful episodes over five zero-shot seeds per task-backbone pair, with $T_{\max} = 6$ min; the hatched extension counts the failed episodes that are solved once the robots’ final policies are frozen and rerun offline, revealing the ability of the P-CaP agents to recover from failures within the episode. Markers (top panel) give median completion times: the total time includes agreement and implementation, whereas the post-deploy time is measured from the last deployment and is the cost of running the protocol alone.

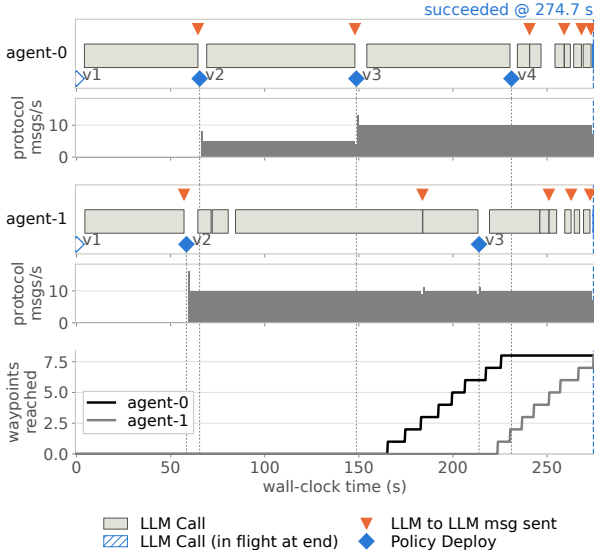


Fig. 5: The timeline of events in a sample episode of P-CaP robots on the Crossed Waypoints task. The robots remain stationary while their LLMs negotiate the protocol. Once the protocol is established the P-CaP agents integrate it into their control policies and deploy to solve the task.

repair a mismatched pair of programs. To reveal whether such repairs happen we freeze the final policies in failed episodes and rerun them offline on the same seed. The hatched bars of Fig. 4 show that a substantial share of the failures is solved by the very programs the robots had already written, so our system has the ability to recover from its mistakes.

Asynchrony and Messaging Patterns. Fig. 5 shows the temporal evolution of a sample episode of the *Crossed Waypoints* task using two agents. We see immediately that the two timelines are fully asynchronous: each robot queries its LLM, sends its messages, and deploys code at its own instants.

TABLE I: Failure modes of P-CaP agents across models and scenarios.

Coordination stage	Failure mode	Failures
Agent loop	Too few LLM turns (inference time)	5
	Unusable LLM output	3
Protocol bring-up	Deployed policy errors	2
	Working policy deployed too late	5
	Never left verification hold	1
Protocol in operation	Moved before peer sent a message	3
	Incorrectly digested peer messages	2
	Protocol up, control failed	3

Both begin by deploying a policy (♦ v1) that keeps the robot stationary while advertising what it perceives, and only once the schema is agreed upon (♦ v3) do we see progress on reaching waypoints. At the same time, the LLMs continue to exchange natural language messages (▼) while their programs run the protocol underneath.

B. Analysis of failure modes

Table I classifies the 24 failed episodes by the stage and mode of the failures in protocol-building process. Ten of the failures were a consequence of establishing a limit to the task completion time. In five instances, the LLMs have implemented a correct protocol but ran out of time to complete the entirety of the task. In another five, the robots suffered from large LLM inference delays which resulted in the LLMs completing too few rounds of reasoning, negotiating and coding to deploy a minimal working solution. Other failures can be attributed to the LLMs instruction following inadequacies. In three failing episodes, outputs from the LLM did not adhere to the format, were not parsed and hence never executed. Three more are bring-up defects, namely deployed policies that raised errors or a verification gate that never opened. Five failures are related to failures of reasoning over

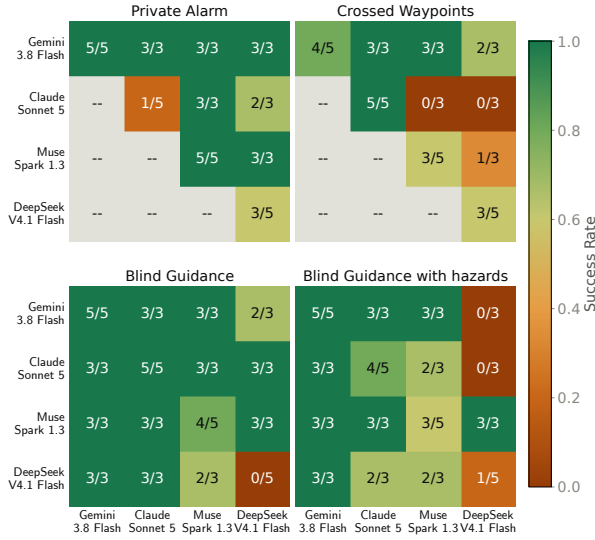


Fig. 6: Cross-play performance of P-CaP agents across backbones. In Private Alarm and Crossed Waypoints scenarios the roles are interchangeable, so only the upper triangle is populated. In the Blind Guidance tasks the roles differ: rows give the backbone of the ground robot, columns that of the aerial robot.

the protocol itself, where a robot started moving before its peer had communicated anything, or implemented a parser that did not match the schema it had just agreed to. The remaining three are failures of the control logic rather than of communication: in one of them the aerial robot of *Blind Guidance with hazards* told its peer to stop beside a moving hazard instead of avoiding it.

C. Cross-play between LLM models

Finally, we pair robots driven by different LLM models, which is the regime an ad hoc encounter we motivate in Section I is likely to pose (Fig. 6). We find that agents that perform well in homogeneous pairs generally perform well heterogeneously, and a strong backbone often pulls a weaker partner up. Exceptions happen, though: two individually competent models occasionally collapse when together, and a weak partner can equally drag a strong one down. Diversity of roles also have an impact in cross-play, and *Blind Guidance with hazards* is a revealing example because its two roles are not interchangeable: DeepSeek V4.1 Flash performs poorly as the guiding (aerial) robot, yet succeeds as the guided (ground) robot. We contend, therefore, that ad hoc competence is not predicted by within-model success, an observation that calls for evaluating LLM-based multi-robot systems in cross-play rather than in self-play.

D. Real-robot demonstration

We finally deploy P-CaP on physical robots. We use *Crossed Waypoints* as the task and Gemini 3.8 Flash as the LLM model, and we run five episodes under the same $T_{\max} = 6$ min budget as in simulation. The system, the prompts, and the skill library are the ones used throughout this section; the only component that changes is the layer beneath `policy.py`, where the Policy Workers now drive Robomaster robots [29]

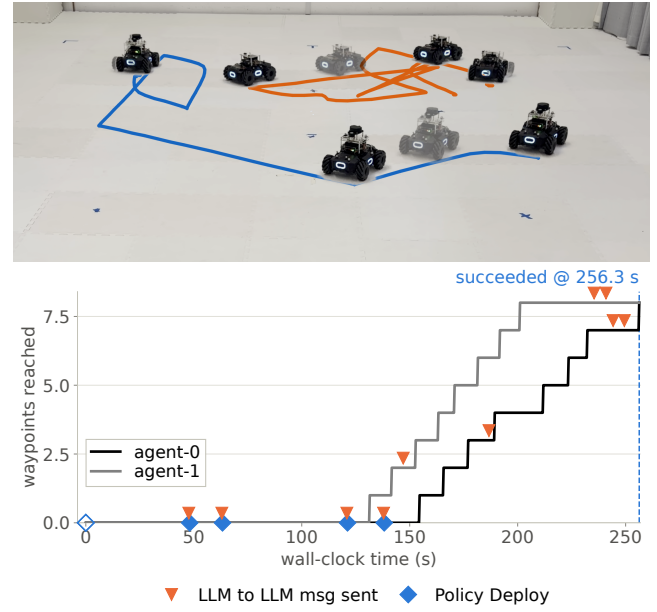


Fig. 7: Trajectories of the robots along with the timeline of their progress on the task, policy deployments and exchanged LLM-to-LLM messages in real-world deployment on the Crossed Waypoints task. The robots successfully negotiate a protocol and deploy it on a real embodiment to solve the task.

in a motion-capture system instead of the simulator. All five episodes succeed. In each, the robots negotiate a protocol for exchanging waypoint coordinates, implement it on both sides, and reach their goals, with a median completion time of 240.6 s, comfortably within the budget and within the 150 s to 320 s band observed in simulation.

Fig. 7 shows the trajectories of a sample episode, and the corresponding timeline. The structure is the one described for simulation in Fig. 5: the robots hold position while advertising what they perceive. Progress towards the waypoints can only begin once the schema is agreed upon and the corresponding policies are deployed, and the LLMs keep conversing while the deployed programs run the protocol underneath. Inference runs against the same cloud endpoints, so the latency λ is unchanged, but the control loop now contends with actuation delay, imperfect state estimation, and the physical execution of the agreed maneuvers, and the protocol must remain valid under all of them. That the agreement holds without any change to the agents suggests that it is the separation of the two loops, rather than any property of the simulator, that enables ad hoc coordination.

VII. CONCLUSIONS

We presented Protocols with Code-as-Policy (P-CaP), a framework that enables robots meeting for the first time during deployment to establish the communication protocols they need in order to coordinate. Talking through the LLM channel alone is too slow for this: when what one robot knows is only useful to another if it arrives within a control period, conversation cannot resolve the partial observability the task imposes. P-CaP therefore separates the two timescales, and each robot

negotiates a message schema with its peers in natural language and then writes its own half of the agreement as code, so that policy and protocol are executed by the same program at the control rate, decoupled from the LLM that keeps reasoning, renegotiating, and redeploying as the task unfolds. Across four asynchronous coordination tasks that require private or complementary information to be acted upon quickly, P-CaP outperforms language-only and communication-free baselines on every task, and succeeds in time-critical scenarios where both fail in every episode. This brings evidence that what makes such tasks solvable is not the ability to talk, but the ability to place the exchanged information inside the control loop. Once a protocol is deployed, the robots complete the task in a time comparable to that of handcrafted expert protocols. Our method not only succeeds in homogeneous settings but also in cross-play, when the robots are driven by different LLMs, which is the regime an ad hoc encounter actually poses.

To isolate the challenge of protocol formation we deliberately simplified several aspects of deployment: the robots are holonomic bodies acting in 2D, their observations are noise-free, and the peer channel is assumed reliable. Relaxing these assumptions is the natural continuation of this work, towards perception stacks based on vision or LIDAR, other embodiments such as manipulators, and skill libraries that also include learned, black-box components.

REFERENCES

- [1] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, “Emergent Abilities of Large Language Models,” *Transactions on Machine Learning Research*, 2022.
- [2] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, “Palm-e: an embodied multimodal language model,” in *Proceedings of the 40th International Conference on Machine Learning, ICML’23, JMLR.org*, 2023.
- [3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” in *Conference on Robot Learning*, 2023.
- [4] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, et al., “Open X-embodiment: Robotic Learning Datasets and RT-X Models: Open X-Embodiment Collaboration,” in *IEEE International Conference on Robotics and Automation*, pp. 6892–6903, 2024.
- [5] Z. Mandi, S. Jain, and S. Song, “Roco: Dialectic multi-robot collaboration with large language models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 286–299, 2024.
- [6] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, “Smart-llm: Smart multi-agent robot task planning using large language models,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 12140–12147, 2024.
- [7] K. Liu, Z. Tang, D. Wang, Z. Wang, X. Li, and B. Zhao, “Coherent: Collaboration of heterogeneous multi-robot system with large language models,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 10208–10214, 2025.
- [8] H. Zhang, W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan, “Building cooperative embodied agents modularly with large language models,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [9] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500, 2023.
- [10] P. Stone, G. A. Kaminka, S. Kraus, and J. S. Rosenschein, “Ad hoc autonomous agent teams: collaboration without pre-coordination,” in *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, AAAI’10*, p. 1504–1509, AAAI Press, 2010.
- [11] X. Liu, P. Li, W. Yang, D. Guo, and H. Liu, “Leveraging Large Language Model for Heterogeneous Ad Hoc Teamwork Collaboration,” June 2024. arXiv:2406.12224 [cs.RO].
- [12] H. Hu, A. Lerer, A. Peysakhovich, and J. Foerster, ““Other-Play” for Zero-Shot Coordination,” in *Proceedings of the 37th International Conference on Machine Learning*, pp. 4399–4410, PMLR, Nov. 2020.
- [13] J. Treutlein, M. Dennis, C. Oesterheld, and J. Foerster, “A New Formalism, Method and Open Issues for Zero-Shot Coordination,” in *Proceedings of the 38th International Conference on Machine Learning*, pp. 10413–10423, PMLR, July 2021.
- [14] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “Progprompt: Generating situated robot task plans using large language models,” 2023.
- [15] L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, L. Fei-Fei, G. Shi, J. Wu, S. S. Sastry, Y. Zhu, K. Goldberg, and L. Fan, “Cap-x: A framework for benchmarking and improving coding agents for robot manipulation,” in *Forty-third International Conference on Machine Learning*, 2026.
- [16] K. Chen, S. Xie, L. Fu, J. Yu, W. Pacini, S. Bajamahal, H. Kim, J. Drake, D. Kim, H. Xue, J. Francis, C. Juette, P. Schaldenbrand, M. Y. Seker, R. Wickramarachchi, U. Yoo, G. Wang, A. Murali, B. Sundaralingam, S. S. Sastry, S. Huang, Y. Zhu, L. J. Fan, and K. Goldberg, “GaP: A Graph-as-Policy Multi-Agent Self-Learning Harness For Variational Automation Tasks,” July 2026. arXiv:2607.05369 [cs.RO].
- [17] S. Borate, B. R. B. V. Pardeshi, and M. Vadali, “Comuros - an llm-based generalizable hierarchical task planning and execution framework for heterogeneous robot teams with event-driven re-planning,” *Frontiers in Robotics and AI*, vol. 13, 2026.
- [18] S. Song, X. Xie, Z. Li, Y. Li, S. Wang, and B. Qi, “Leveraging Adaptive Group Negotiation for Heterogeneous Multi-Robot Collaboration with Large Language Models,” Dec. 2025. arXiv:2602.06967 [cs.RO].
- [19] W. Ji, H. Chen, M. Chen, G. Zhu, L. Xu, R. Groß, R. Zhou, M. Cao, and S. Zhao, “GenSwarm: Scalable Multi-Robot Code-Policy Generation and Deployment via Language Models,” *npj Robotics*, vol. 4, p. 5, Jan. 2026.
- [20] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, “Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4311–4317, 2024.
- [21] V. Strobel, M. Dorigo, and M. Fritz, “Llm2swarm: Robot swarms that responsively reason, plan, and collaborate through llms,” in *NeurIPS 2024 Workshop on Open-World Agents (OWA-2024)*, 2024.
- [22] Z. Li, W. Zhao, and J. Pajarinen, “Closed-Loop Vision-Language Planning for Multi-Agent Coordination,” May 2026. arXiv:2502.10148 [cs.AI].
- [23] H. Shi, S. Ye, K. M. Guerrero, J. Shen, Y. Yin, D. Khashabi, C.-M. Huang, and T. Shu, “Safe and Interpretable Multimodal Path Planning for Multi-Agent Cooperation,” Feb. 2026. arXiv:2602.19304 [cs.RO].
- [24] R. Doshi, T. Gao, A. Chen, C. Finn, and J. Bohg, “CHORUS: Decentralized Multi-Embodiment Collaboration with One VLA Policy,” June 2026. arXiv:2606.12352 [cs.RO].
- [25] Google DeepMind, “Gemini 3.8 flash: Model card,” Model card, Sept. 2026.
- [26] Anthropic, “Claude sonnet 5 system card,” system card, Anthropic, June 2026.
- [27] Meta, “Muse spark 1.3,” Official model documentation, Sept. 2026.
- [28] DeepSeek-AI, “Deepseek-v4.1-flash: Pushing the limits of KV cache compression,” technical report, DeepSeek-AI, Sept. 2026.
- [29] J. Blumenkamp, A. Shankar, M. Bettini, J. Bird, and A. Prorok, “The Cambridge Robomaster: An Agile Multi-Robot Research Platform,” in *International Symposium on Distributed Autonomous Robotic Systems*, pp. 439–456, Springer, 2024.